Antikit-8 | Gerenciamento de memória

Relatório técnico de implementação e testes • Sistemas Operacionais • 28/10/2026

Grupo: Heitor Viana Ferreira, Isabely Carvalho Lima e Fabrício Rodrigues dos Santos. Os responsáveis por módulos estão indicados na página 6.

Objetivo e resultado

O simulador implementa paginação sob demanda com tamanhos alternados, RAM compartilhada de 256 KiB e algoritmo Aging adaptado ao Swap térmico. Os arquivos hardware_specs.c e hardware_specs.h foram mantidos byte a byte. A solução usa uma tabela autoritativa de páginas por processo e verifica a correspondência com os quadros físicos após cada operação relevante.

Nos traces sintéticos executados, a política eficiente manteve o pico térmico em 37 °C para dez novas leituras e em 62 °C para dez novas escritas. A comparação sem controle chegou a 172 °C. A espera térmica é registrada separadamente: o ganho em ciclos de E/S não implica ausência de latência.

Tradução assimétrica

Cada par de páginas ocupa 4.096 + 8.192 = 12.288 bytes. Para um endereço a, calcula-se q = a / 12.288 e r = a % 12.288, usando divisão inteira. Se r < 4.096, VPN = 2q e offset = r. Caso contrário, VPN = 2q + 1 e offset = r - 4.096.

O espaço de 1.048.576 bytes contém 85 pares completos e uma página par de 4 KiB: **171 páginas, de VPN 0 a VPN 170**. O endereço fora desse intervalo é rejeitado antes de chamar a MMU.

Endereço virtual	VPN	Offset	Interpretação
12.287	1	8.191	Último byte da página ímpar
12.288	2	0	Início da próxima página par
1.048.575	170	4.095	Último byte do espaço virtual

Organização da RAM e page faults

A RAM tem 64 quadros de 4 KiB. Uma página par usa um quadro e uma ímpar usa dois contíguos. frame_number identifica o primeiro quadro; endereço físico = frame_number × 4.096 + offset. Se não houver bloco contíguo, a substituição remove vítimas até haver espaço. Memória livre não contígua pode exigir mais de uma remoção.

Quando mmu_access_memory retorna false para um acesso válido, o SO conta um fault, obtém espaço, faz Page Out de vítimas sujas, mapeia a página e repete o acesso. A MMU registra R, M e o tempo do acesso efetivamente concluído. O simulador modela residência e métricas, sem armazenar conteúdo de bytes em RAM ou disco.

Aging e controle térmico

Algoritmo de substituição

Cada página possui um contador Aging de oito bits, inicialmente zero. A cada cinco chamadas globais da MMU, para cada página residente: age = (age >> 1) | (R ? 128 : 0); depois o SO limpa R. O histórico é estado legítimo do algoritmo e não uma tabela paralela que restaura referências apagadas pelo hardware.

A pontuação da vítima é age + 256R + 512M. O menor valor vence. Assim, qualquer página limpa precede uma suja; dentro da classe, R atual protege acessos desde a última amostragem, e Aging ordena o histórico. Empates seguem a ordem de criação dos processos e a VPN crescente. Na baseline, elimina-se o termo 512M e não se espera por resfriamento.

A preferência absoluta por limpas reduz Page Outs neste teste, mas pode expulsar páginas limpas recentes e aumentar faults em outras cargas. Não se afirma optimalidade global. O fading pode diminuir a importância estimada de páginas usadas, pois sua modificação de R é respeitada antes da amostragem.

Comportamento do motor fornecido

No mesmo milissegundo do último Page Out, o motor aquece 15 °C. Em outro milissegundo, ele primeiro resfria 0,5 °C por ms desde last_swap_ms, com piso em 25 °C, e depois aquece **12 °C**. Essa segunda regra está no código fornecido, embora o enunciado destaque o aumento de 15 °C.

Acima de 65 °C, o custo é 1.000×2^{n}, com n = floor((T - 65)/5) + 1. Logo, a regra exata do código também determina os casos de borda. Não foi alterado o custo do professor nem acrescentado um custo de Page In inexistente na API.

Como a espera evita superaquecimento

Antes de escrever uma vítima suja, o SO estima a temperatura depois da operação. Se o limite seria ultrapassado, avança o relógio de execução e repete a estimativa. O timestamp de chegada do trace é preservado; o processo espera. Nenhuma espera real do computador é necessária.

O resfriamento desse intervalo é aplicado pelo próprio hardware_trigger_page_out, que chama hardware_cool_down. Chamar cool_down externamente antes desse Page Out contaria o mesmo intervalo duas vezes. Durante intervalos sem escrita, o CSV mostra a projeção passiva da temperatura; ao terminar, o resfriamento pendente é materializado uma única vez.

Contabilização sem confundir medidas

Acessos do trace são operações lógicas. Chamadas da MMU incluem tentativas e retries; para o fluxo implementado, chamadas = acessos + faults. O fading segue o contador do hardware. Ciclos de CPU totais incluem um ciclo por chamada e os custos de Page Out. Ciclos de E/S são obtidos por diferença do contador ao redor de cada Page Out. Esperas são informadas em ms, sem conversão inventada para ciclos.

O motor usa deslocamento de um int no cálculo da penalidade. A baseline rejeita uma operação que produziria deslocamento indefinido em temperaturas extremas; os testes apresentados não atingem esse limite. A política eficiente não chega à faixa de throttling.

Teste térmico | resultados medidos

Preenchimento: 32 páginas ímpares de 8 KiB ocupam exatamente 262.144 bytes. Depois, dez páginas ímpares novas são acessadas com chegada em 200 ms. Cada execução tem 42 acessos, 42 faults e 84 chamadas à MMU. A semente é srand(1); os valores abaixo vêm dos logs incluídos.

Carga / política	Page Outs	Pico °C	Ciclos E/S	Espera ms
Leituras / eficiente	1	37	1.000	0
Leituras / baseline	10	172	4.793.492.000	0
Escritas / eficiente	10	62	10.000	172
Escritas / baseline	10	172	4.793.492.000	0

Na carga de leitura, após a primeira vítima suja, a página recém-carregada fica limpa e pode ser substituída. Portanto o ganho não depende de adiar dez escritas: nove Page Outs deixam de existir. Em RAM inicialmente mista, a preferência por limpas elimina todos os Page Outs desta mesma sequência.

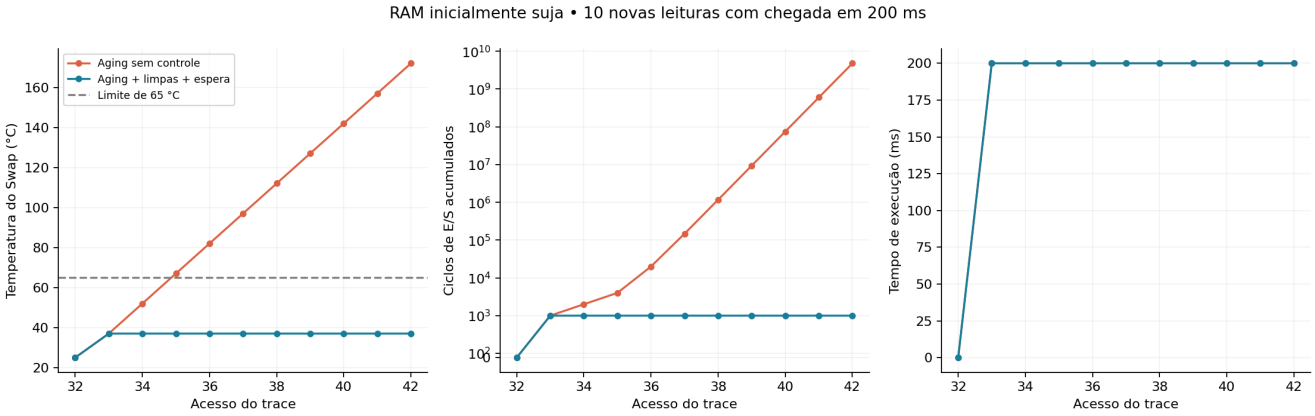


Figura 1. RAM inicialmente suja, novas leituras. O eixo de ciclos usa escala symlog para mostrar desde zero até bilhões de ciclos. Os timestamps de execução permanecem em 200 ms durante as dez novas leituras.

Na carga de escrita, as páginas novas também ficam sujas. A política eficiente executa dez Page Outs a custo base, usando espera virtual. O último acesso termina em 372 ms, embora os dez tenham chegado em 200 ms. A baseline termina a sequência com timestamp simulado 200 ms e custo muito maior; o hardware não converte seus ciclos de CPU em avanço do relógio.

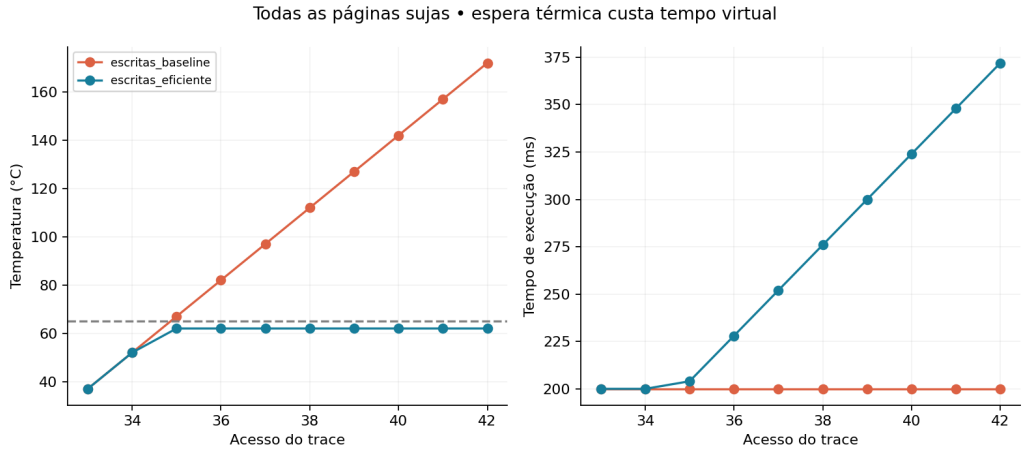


Figura 2. Todas as páginas sujas: a comparação explicita a troca entre temperatura, ciclos de E/S e tempo virtual de espera. Não representa tempo real de execução do computador hospedeiro.

Auditoria e resiliência ao fading

O que o hardware realmente faz

A cada quinta chamada, a MMU sorteia rand() % num_entries no processo corrente. Se a página sorteada estiver ausente, nada muda. Se estiver presente, R é zerado. O sorteio não é limitado às páginas residentes e não percorre as tabelas de todos os processos. O simulador preserva essas regras do código fornecido.

Para observar o alvo, somente a compilação de hardware_specs.c substitui a chamada rand por antikit_observed_rand, via definição do compilador. O observador chama o rand original exatamente uma vez, devolve o mesmo valor e guarda o resultado para o log. Não muda o arquivo original, o sorteio ou as regras da MMU.

Ao retornar da MMU, o programa imprime FADING e imediatamente AUDIT. A API não oferece um ponto de observação dentro da função após limpar R. Por isso o log registra R anterior e R no retorno: se o alvo é a própria página acessada em um hit, R pode ter sido zerado e já restaurado para 1 pelo próprio hardware. Não se apresenta essa observação como snapshot do instante intermediário.

Três valores de auditoria e uma verificação estrutural

Campo	Conteúdo e finalidade
map	FNV-1a de PID, ordem das entradas, present e frame_number. Atende ao checksum do mapeamento.
estado	Inclui também referenced, modified e last_access_t. Permite observar alterações além do mapeamento.
acumulado	Encadeia o checksum map e a sequência dos eventos. Ajuda a acompanhar o histórico do log.
Consistência	Confere limites, sobreposição, PTEs ausentes sem quadro e correspondência bidirecional entre PTEs e quadros.

FNV-1a é um checksum simples, não um mecanismo criptográfico. Não impede que código malicioso forje dados e logs. Em particular, present/frame_number podem permanecer iguais quando R muda; não é correto exigir que esse hash detecte sozinho o fading. O checksum de estado e a verificação estrutural complementam o requisito sem prometer detecção de qualquer fraude.

Evidência do teste 3

No trace de 300 acessos foram observadas 573 chamadas à MMU e 114 sorteios de fading, nos contadores 5, 10, ... 570. O teste automático verifica toda essa sequência e que cada linha FADING é seguida por AUDIT. Há sorteios em páginas ausentes e alterações reais de R.

Exemplo do log incluído: mmu=305, pid=1, vpn=24, presente=1, R_antes=1, R_retorno=0, efeito=zerado_pelo_hardware. O registro seguinte recalcula os checksums. O teste de corrupção proposital altera o dono de um quadro e confirma que a auditoria encerra com erro RAM e PTE divergentes.

Não há cópia persistente de R/M para guiar decisões. O pequeno snapshot de R usado pelo observador serve apenas para explicar o evento e é descartado após a chamada; o algoritmo consulta a mesma tabela entregue à MMU.

Fragmentação e mapa ASCII

Definição mensurável de memória útil

O trace de endereços, sozinho, não informa o tamanho útil de uma alocação: visitar um byte de uma página não significa que apenas aquele byte seja necessário. Foi adotada a coluna opcional bytes_uteis_da_pagina, que declara um prefixo útil. Sem essa coluna, a página é considerada inteiramente útil. Essa premissa é explícita e deve ser alinhada ao formato oficial se outro trace for fornecido.

Para cada página residente, desperdício = tamanho da página - bytes úteis. A porcentagem pedida sobre a RAM é 100 × soma(desperdício) / 262.144. A média final é aritmética sobre os acessos válidos do trace, inclusive o último bloco incompleto. Também se informa desperdício / bytes alocados, com denominador diferente.

Página	Útil	Desperdício	% da página	% da RAM
Par, 4 KiB	1 KiB	3 KiB	75%	1,171875%
Ímpar, 8 KiB	1 KiB	7 KiB	87,5%	2,734375%

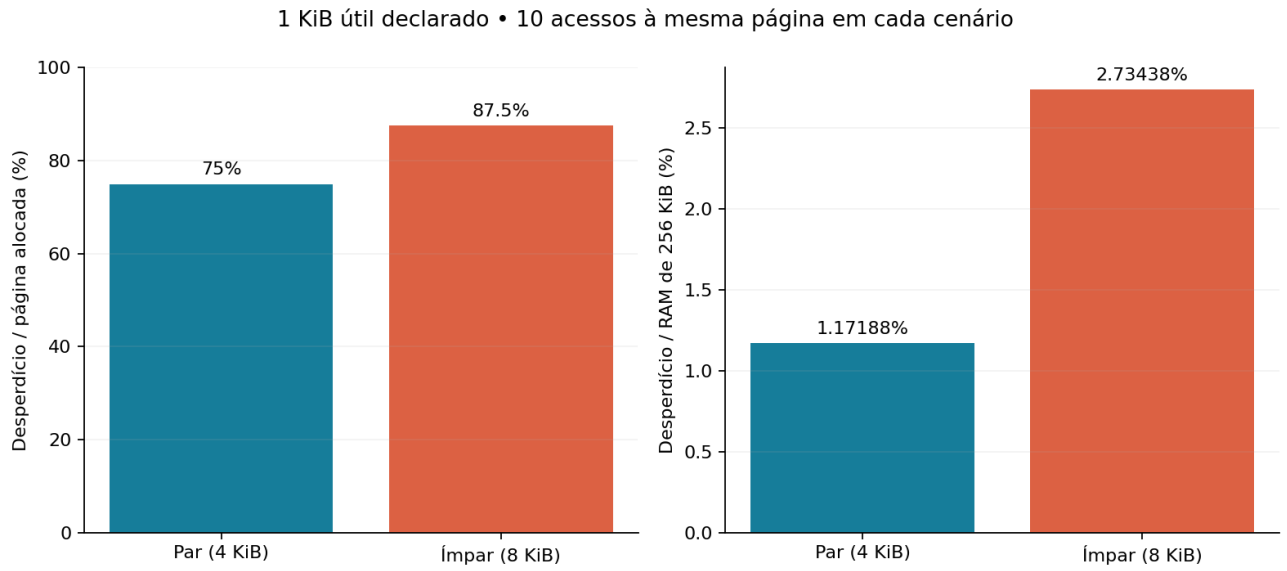


Figura 3. Cada cenário faz dez acessos à mesma página e declara 1 KiB útil. Assim, a ocupação fica constante e os valores do gráfico também são as médias dos cenários. A diferença de desperdício é de 4 KiB.

Leitura do visualizador

O mapa é emitido a cada dez acessos válidos; comentários e linhas vazias não contam. Há 256 caracteres, um por KiB, distribuídos em oito linhas. U indica uma região com bytes úteis, F uma região inteiramente desperdiçada e ponto uma região livre. O modo --pause aguarda Enter.

Exemplos de prefixos: [UFFF...] para uma página par com 1 KiB útil e [UFFFFFFF...] para uma ímpar com 1 KiB útil. Para quantidades não múltiplas de 1 KiB, um bloco parcialmente útil aparece como U, mas as métricas continuam exatas em bytes.

As páginas de 8 KiB precisam de dois quadros contíguos. Buracos entre alocações podem dificultar encontrar esse bloco, mas continuam livres e não são contados como fragmentação interna. Esse efeito de contiguidade é distinto do desperdício dentro de uma página alocada.

Validação, reprodução e divisão do grupo

Testes executados

A suíte percorreu todos os 1.048.576 endereços válidos, comparando a fórmula de tradução com a página modificada pela MMU do professor. Também verificou os três limites pedidos, rejeição de endereços inválidos, isolamento entre PIDs, RAM sob pressão, parser estrito, custos de E/S, controle térmico, sequência do fading e os valores exatos de fragmentação.

Um teste adicional usa 600 acessos em oito PIDs com páginas de 4 e 8 KiB, leituras e escritas. As invariantes de quadros e tabelas são verificadas durante a execução. A compilação C11 usa Wall, Wextra, Wpedantic e Werror.

AddressSanitizer e UndefinedBehaviorSanitizer aprovaram a suíte com detect_leaks=0. LeakSanitizer não pôde executar por restrições de acesso a /proc/ptrace neste ambiente. Portanto a ausência de vazamentos não foi certificada por essa ferramenta. Na execução normal, o bloco principal alocado com calloc é liberado e os arquivos são fechados; erros fatais podem encerrar imediatamente.

Reprodução

Em terminal Linux/WSL na pasta do projeto:

```
make
```

```
make test
```

```
make results
```

```
./antikit8 traces/termico_sujo.txt --csv eficiente.csv
```

```
./antikit8 traces/termico_sujo.txt --naive --csv baseline.csv
```

```
./antikit8 traces/fragmentacao_impar.txt --pause
```

O executável requer compilador C e Make; testes usam Python 3. Gráficos requerem matplotlib. Este relatório pode ser regenerado com python3 docs/gerar_relatorio.py (reportlab). Os arquivos de hardware possuem checksums SHA-256 em hardware/SHA256SUMS. Sequências de rand podem variar entre bibliotecas C; os logs fornecidos registram esta execução.

Responsabilidades

Integrante	Responsabilidade	Arquivos
1 - Heitor Viana Ferreira	Tradução, RAM e consistência física	src/memory.c
2 - Isabely Carvalho Lima	Aging, escolha de vítimas e controle térmico	src/replacement.c
3 - Fabrício Rodrigues dos Santos	Parser, auditoria, mapas e métricas	src/parser.c; src/audit.c; scripts de análise

A integração está em main.c e simulator.h.

Referências e limites

Tanenbaum, Andrew S. Sistemas Operacionais Modernos, capítulo 3, referência indicada pelo enunciado; edição e páginas não informadas. Fontes normativas desta implementação: enunciado e arquivos hardware_specs.h / hardware_specs.c fornecidos. Os traces e resultados são experimentos sintéticos deste pacote, pois não foi anexado memory_trace.txt oficial.